IEEE Transactions on Machine Learning in Communications and Networking

Received 26 September 2024; revised 9 March 2025 and 12 May 2025; accepted 22 July 2025.
Date of publication 31 July 2025; date of current version 5 August 2025.

The associate editor coordinating the review of this article and approving it for publication was E. Bou-Harb.

Digital Object Identifier 10.1109/TMLCN.2025.3594659

A Data-Driven Approach to Mitigate Evolving Volumetric Attacks in Programmable Networks

MUHAMMAD SAQIB[1] (Graduate Student Member, IEEE), HALIMA ELBIAZE[1] (Senior Member, IEEE), AND ROCH H. GLITHO[2] (Life Senior Member, IEEE)

[1]Department of Computer Science, Université du Québec à Montréal, Montreal, QC H3C 3J7, Canada
[2]Concordia Institute of Information Systems Engineering, Concordia University, Montreal, QC H3G 1M8, Canada
CORRESPONDING AUTHOR: M. SAQIB (saqib.muhammad@courrier.uqam.ca)

ABSTRACT In-network machine learning (ML) offers a cutting-edge approach for promptly detecting malicious traffic. Existing methods often rely on one-size-fits-all ML models that fail to adapt to evolving attack traffic patterns, leading to a time-consuming and labor-intensive process for updating ML model from the control to the data plane. To address these limitations, we propose an automated, data-driven method for identifying novel malicious traffic patterns and updating ML model seamlessly in programmable networks. The proposed method sets drift detection thresholds based on baseline performance from historical (i.e., training) data and uses these thresholds to detect anomalies in unseen (i.e., testing) data. We continuously adjust the thresholds to accommodate data distribution changes and in-network inference results while minimizing sensitivity to minor fluctuations. We evaluate the proposed method using two intrusion detection datasets, CICIDS2017 and UNSW-NB15. The experimental results demonstrate its efficacy in safeguarding against evolving volumetric attacks. Additionally, we compare the conventional model performance-based drift detection method with an adaptive monitoring window-based approach, highlighting the latter's advantage in balancing drift detection efficacy and minimizing its adaptation impact, i.e., disruptions to normal network traffic are reduced by an average of 20%. The adaptive method dynamically adjusts the drift monitoring window size to adapt to the characteristics of the unseen traffic patterns.

INDEX TERMS Network intrusion detection, machine learning, programmable networks, P4.

I. INTRODUCTION

MACHINE learning (ML) has proven to be a valuable tool for traffic classification and network security [1]. Network-oriented ML tasks are traditionally executed on servers or middleboxes [2]. However, recent advancements in programmable data planes (PDP) have introduced new possibilities for cost-effective, flexible, high-performance network security [3]. Network administrators can directly program packet processing logic using network programming languages, such as Programming Protocol-independent Packet Processors (P4) [4], enabling the offloading of ML operations to the network data plane. This approach allows for in-network ML [5], which strategically divides the training and inference phases between the control and data planes for efficient network security.

In-network inference is a key enabler for developing efficient and accurate attack detection solutions [6], [7]. However, the evolving landscape of malicious activities introduces variations in the network traffic patterns using different protocols, attack vectors, and the data distribution of malicious traffic [8]. Such dynamic movement of network traffic properties causes *concept drift* [9], which can be a sign of anomalies because it represents a departure from the established patterns and statistical properties on which an ML model has been trained [10]. As the data distribution shifts over time, it may indicate the emergence of new or unusual patterns, affecting the relevance of input features (i.e., port number and statistical properties of packet size) of a trained ML model and their classification decision boundaries. Such a shift in data distribution may ultimately lead to decreased accuracy because the model must adapt to evolving patterns in the data for accurate prediction [11]. Therefore, adaptive ML models that recognize and respond to changing traffic patterns are crucial for an effective threat defense [12].

One of the major limitations of existing in-network attack detection methods is the one-shot learning of ML models [6], [7], [13], [14], [15], [16], [17]. The mapping rules of these models are initially defined based on historical data at the control counterpart following preprocessing and hyperparameter tuning. The resulting rules are then mapped onto the data plane for online inference. Because the entire ML operation is strategically divided into control and data planes, monitoring individual instances of malicious activity, identifying previously unseen malicious traffic patterns using preprocessing and hyperparameter tuning, and updating the ML model from the control to the data plane are infeasible in such a pragmatic framework. These results in frequent updates to the ML model, which can cause significant network overhead and disrupt normal network traffic. In addition, in-network ML updates are more complex than server-based ML, and conducting a hitless update process with minimal impact on normal packet forwarding performance is challenging [18]. Therefore, it is essential to integrate the functionalities of the control and data planes to execute in-network ML updates that address emerging anomalies with minimal disruption caused by the updates. However, existing works still need to address the challenge of detecting and seamlessly adapting to unseen malicious traffic patterns in programmable networks to protect against evolving attacks.

This study aims to automate drift detection and adaptive ML model updates in a programmable network, ensuring seamless adaptation to changing traffic patterns and mitigating evolving volumetric attacks. The framework employs a statistical drift detection mechanism that monitors feature distribution shifts in network traffic, triggering adaptation process from the control plane to the data plane. Specifically, drift detection and adaptation logic are distributed across the control and data planes. The control plane trains the ML model on historical data and loads the resulting decision rules into the *Match-Action Tables (MATs)* of the PDP for real-time inference. The data plane employs a lightweight monitoring sketch for real-time feature extraction from traffic flows. The extracted features are utilized for in-network inference, where the system applies ML rules using *MATs*. When inference performance deteriorates, the data plane logs the extracted features to the control plane for drift verification and adaptation. The control plane subsequently applies drift detection method to assess distribution shifts. If drift is detected, it initiates ML model update, refines classification rules, and loads them into the *MATs* without significantly disrupting normal network processing. This approach enables efficient, real-time adaptation to concept drift, ensuring that the network maintains detection accuracy even as attack patterns evolve.

We first validate the existence of drifts and their implications using two intrusion detection datasets: CICIDS2017 [19] and UNSW-NB15 [20]. Subsequently, we evaluate the efficacy - the ability to detect drifts under defined conditions - of the proposed data-driven method. The experimental results reveal the efficacy of the adaptive method for safeguarding against evolving volumetric attacks. We further compare the conventional model performance-based drift detection method with that enabled by an adaptive monitoring window, emphasizing the latter's superiority in balancing the efficacy of drift detection and its adaptation impact on the disruption of normal network traffic. This balance is achieved by dynamically adjusting the drift monitoring window size to better adapt to the characteristics of the unseen data.

The key contributions of this work are as follows.

- We propose a self-adaptive traffic classification method that detects and responds to evolving volumetric attacks. By continuously monitoring traffic feature distributions and detecting concept drift, the system updates the ML model to maintain classification effectiveness in programmable networks.
- The framework leverages the interplay between the control and data planes for efficient adaptation. The data plane extracts traffic features and logs performance degradation, while the control plane verifies drift using statistical methods and updates the ML model to refine classification rules. This design helps balance adaptation with operational overhead.
- To further optimize adaptation, we introduce an adaptive windowing strategy that dynamically adjusts the frequency of retraining. This mechanism ensures that updates occur only when necessary, reducing unnecessary retraining while maintaining detection performance.

The remainder of this paper is organized as follows. Section II provides an overview of the related work. Section III introduces the proposed method. Section IV discusses the validation methodology, and Section V presents the experimental results. Finally, Section VI concludes the study.

II. RELATED WORK

This section comprehensively provides an overview of the current volumetric attack detection approaches in programmable networks.

We position our approach with state-of-the-art in Table 1. Our method is designed to meet the following key requirements: learning-based traffic classification, line-rate processing, drift detection, and runtime updates. These design requirements are essential for building an adaptive method that addresses the dynamic nature of volumetric attacks in programmable networks.

The first category of prior art consists of methods that extract valuable flow information in the data plane to support

TABLE 1. Summary of the related work.

Category	Reference	Learning based	Line speed	Drift detection	Runtime updates	Comment
1st	[21], [22]	✓	x	x	x	Classification in control plane
2nd	[23], [24]	x	✓	x	✓	Threshold driven traffic filters
3rd	[25]	x	✓	x	x	Predefined threshold based rules
	[16]	✓	✓	x	x	-
	[3], [6], [15], [17]	✓	✓	x	✓	-
	[26]–[28]	✓	✓	x	✓	-
	This study	✓	✓	✓	✓	Learning based threshold adjustment

1st: collect flow information in the data plane and traffic classification on the control plane; 2nd: like 1st category with classification logic based on threshold-driven traffic filters; 3rd: embed the learning-based models in the data plane.

broader applications. For example, Othmane et al. [22] and FlowLens [21] use programmable switches to collect flow-distribution information, which is subsequently used by the control plane for learning-based traffic classification. While these methods meet the need for learning-based classification, they rely on the control plane for decision-making, limiting their ability to process data at line speed and respond to real-time network conditions.

The second category includes Poseidon [23] and Jaqen [24], both of which adopt designs similar to FlowLens [21]. However, the collected flow information is directly processed in the data plane to identify volumetric attacks at the line rate. These approaches fulfill the line-rate processing requirement but rely on threshold-driven filters rather than adaptable learning-based models, making them less effective in scenarios where handcrafted filters cannot accurately represent traffic analysis logic.

The third category of prior art takes a step further to realize intelligence in the data plane. The concept is referred to as in-switch inference, which brings remarkable benefits to the network, i.e., reducing latency and increasing throughput [3]. As a result, there has been growing interest in integrating the output of ML algorithms into the data plane for the classification of network traffic at an early stage. Ding et al. [25] propose an in-network DDoS victim identification method using a sketch-based data structure that estimates the per-destination flow cardinality using a fixed threshold-based rule to directly identify victims in the data plane of the network. Although this approach supports line-rate processing, it lacks learning-based traffic classification, limiting its ability to adapt to evolving traffic patterns.

More advanced methods, such as those proposed by Musumeci et al. [16], BACKORDERS [17], and NetBeacon [6], attempt to overcome these limitations by incorporating learning-based models directly into the data plane for early-stage volumetric attack detection. Coelho and Schaeffer-Filho [17] demonstrate how different ML classifiers can improve accuracy and reduce inference time. Similarly, BACKORDERS [18] focuses on increasing classification accuracy and efficiency. NetBeacon [7] advances the state-of-the-art by introducing a multi-phase sequential model architecture that performs dynamic packet analysis at near-line rate, meeting both learning-based classification and line-rate processing requirements. Furthermore, these methods enable real-time updates of ML model directly within the data plane, enhancing adaptability to evolving traffic patterns.

Other recent works on in-switch inference include IIsy [28], a hybrid approach that maps ML-based classification models to programmable switches using tree-based classifiers and a backend-assisted decision-making process to reduce computational overhead on the switch. Additionally, a learning-based ransomware mitigation system [26] leverages Random Forest models within programmable switches for detecting and mitigating ransomware activity without relying on deep packet inspection. The In-Network Classification (INC) method [27] extends this paradigm by incorporating a bagging ensemble to detect botnet infections in real-time within Tbps traffic flows, forwarding identified threats to a controller for further clustering and inference.

One of the primary areas for improvement of the current art is the one-shot learning of ML models. Existing approaches involve experimentally defining mapping rules for substantial historical data through preprocessing and hyperparameter tuning, after which the learned rules are mapped to the data plane for online inference. However, the attacking traffic landscape is continually evolving, which can compromise the relevance of the input features and their classification decision boundaries [29], [30]. As a result, even though a one-size-fits-all ML model may exhibit initial accuracy, it can quickly become outdated as traffic patterns change. Therefore, it is crucial to implement a learning-based model that can efficaciously adjust to fluctuations in traffic patterns and continuously classify traffic flows with the utmost accuracy.

The ability to update ML models in the *MATs* of programmable network devices is considered in several studies [3], [6], [13], [15], [17]. However, the effectiveness of the inference process relies on the manual tuning of ML models, from control to the plane, which requires a predefined set of mapping rules for specific scenarios. For example, techniques for detecting anomalies require defining an optimal set of features and their classification decision boundaries (i.e., threshold values). Consequently, administrators

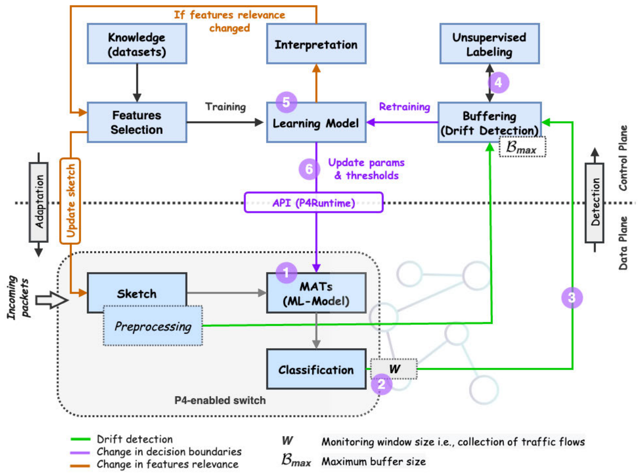

FIGURE 1. **System design.**

and security researchers must invest significant time and effort in investigating various parameters and determining the most suitable ones for evolving attack types. This process is time-consuming and labor-intensive, hindering the adoption of these techniques in modern production networks [5]. Therefore, it is essential to achieve automatic parameter tuning of entire ML operations in programmable networks to seamlessly adapt to changing traffic patterns and maintain flow identification accuracy. However, existing works have not addressed the need to detect and seamlessly adjust to unseen malicious traffic patterns in programmable networks to protect against evolving attacks.

Our work falls into the third category (in-switch inference) but extends it further by introducing adaptivity to traffic classification. Unlike existing studies, which rely on static ML models, our framework automates drift detection and adaptive ML model updates. By integrating automated drift detection and ML model adaptation, our study addresses the critical limitations of existing in-switch ML methods, making it more adaptive, efficient, and suitable for evolving volumetric attack detection in programmable networks.

III. SYSTEM DESIGN

This section introduces the proposed adaptive approach to identify and seamlessly adapt to unknown traffic patterns in programmable networks. A high-level overview of the proposed framework is presented in Fig. 1. The control plane trains the ML model using historical data and updates it based on unknown traffic patterns from unseen (i.e., future) data. The data plane maintains a customized monitoring sketch for extracting features from traffic flows and uses the updated ML model rules for online inference. In the following subsections, we present the details of the distributed entities across the control and data planes, and the integrated data-driven method used to automate drift detection and adaptation processes. The source code of the proposed framework has been made publicly[1].

[1] https://github.com/em-saqib/AdapNet-IDS

A. CONTROL PLANE

The control plane consists three essential logical components of offline learning: *knowledge*, *learning*, and *interpretation*. The following subsections provide a detailed discussion of each component.

1) KNOWLEDGE

The datasets used in this study contain both *benign* and *malicious* traffic patterns. The knowledge is derived from two primary sources: historical data (the initial data) and future data, representing unseen traffic patterns. The historical data is utilized for the initial learning phase, while the unseen data supports the continuous learning of emerging traffic patterns.

To learn from the unseen data, the control plane processes the encapsulated messages generated by the data plane, which contains the extracted features information of the newly received traffic flows. Specifically, when the system detects a drift warning, the control plane begins to *buffer* the extracted feature information and applies an *unsupervised learning* algorithm to label the buffered instances for model retraining. Feature selection is applied to ensure only the most relevant features are retained for classification, optimizing model performance. Meanwhile, unsupervised labeling assigns labels to the buffered instances without requiring human intervention, making the retraining process more efficient.

2) LEARNING

The control plane uses a *learning module* to gain insights into benign and malicious traffic patterns from historical and unseen data. The dataset, denoted as D, consists of packets from subflows and is divided into two subsets: a training set, D_{tr}, and a testing set, D_{ts}. The *learning module* initially trains the ML model on D_{tr}. Subsequently, it embeds the learned rules, represented as *Match-Action (MA)* rules, into *MATs* within a P4-enabled programmable switch [31] for real-time inference. The embedding process is facilitated by a control plane application programming interface (API) called P4Runtime [32].

3) INTERPRETATION

The attackers aim to increase discrepancies between the training and testing data by modifying traffic characteristics, which can shift the relevance of input features and classification decision boundaries. Given limited memory constraints and the inability to support complex operations in PDP [33], using a classifier with a minimal set of features is preferable. Therefore, it is important to establish and maintain an optimal set of features to enable efficient and precise classification. In light of these considerations, we consider it crucial to employ *explainability* technique with a focus on feature importance to enhance the transparency and comprehension of the decision-making processes of ML model. This helps to preserve the most pertinent features for online inference.

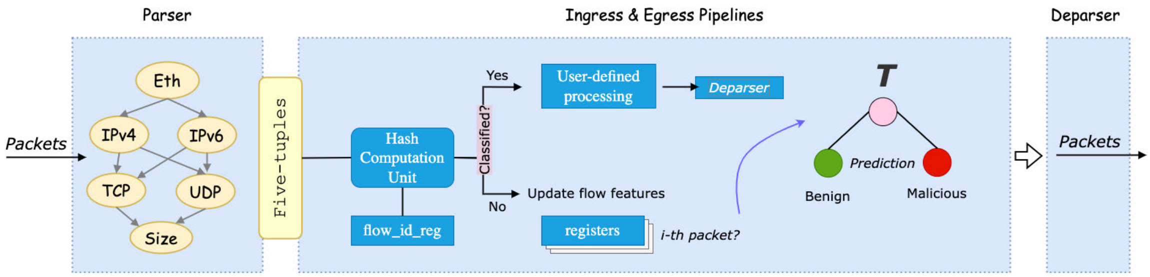

FIGURE 2. Online inference process.

B. DATA PLANE

The data plane is responsible for extracting flow-level features and applying inference logic to detect and mitigate attacks. The general process of online inference is illustrated in Fig. 2.

To support near line-rate classification, the data plane employs in-band feature extraction, enabling immediate retrieval of relevant flow statistics without relying on offline packet captures. This process is handled through protocol-independent processing pipelines in the PDP, implemented in P4 [4]. The parser module operates as a state machine, extracting key flow identifiers such as the five-tuple (source IP, destination IP, source port, destination port, protocol) and packet size from packet headers. These values are stored in the metadata of the switch pipeline for further processing.

To efficiently track flow-level statistics, the data plane maintains a *Flow-Register Sketch* using SRAM registers. A *flow_id* register maps each active flow to a specific index, which is then used to update dedicated per-flow registers, each responsible for storing a distinct feature type (e.g., maximum, minimum and mean of packet sizes). When a packet arrives, its five-tuple is hashed to determine its flow index, and the relevant registers are updated accordingly.

Once a flow reaches the predefined classification threshold, the switch computes aggregate statistics and applies *MATs* to classify the flow. Packets belonging to classified flows are either forwarded or dropped at near line-rate based on the decision outcome. When classification accuracy begins to degrade due to unseen traffic patterns, the data plane logs preprocessed feature information and transmits it to the control plane for further analysis and model updates. This adaptive mechanism ensures continuous learning and resilience against evolving volumetric attacks.

C. INTEGRATED DATA-DRIVEN METHOD

The control plane offers ample storage and computational resources, but its use introduces latency and throughput overheads. In contrast, the data plane excels in providing near line-rate inference capabilities for swift decision-making [3]; however, it has limitations in supporting computational resources and operations [33]. We propose an integrated method that merges the strengths of both planes to safeguard the network against evolving volumetric attacks. Specifically, we leveraged the capability of the control plane to determine drift detection thresholds based on the baseline performance derived from historical data, continuously updating the thresholds during the online inference phase in the data plane. The mathematical formulation of the drift detection parameter definition and algorithms for continuous learning are discussed in the following subsections.

1) DEFINING BASELINE PERFORMANCE

The dataset D comprises n labeled data instances, each characterized by a k-dimensional feature vector x_i and a binary label y_i. It is expressed as

$$D = \{(x_i, y_i)\}_{i=1}^{n},$$

where $x_i = (x_{i1}, x_{i2}, \ldots, x_{ik}) \in \mathbb{R}^k$ represents the feature vector of i-th instance, and $y_i \in \{0, 1\}$ signifies the corresponding label indicating whether the instance is benign (0) or malicious (1).

As new traffic patterns emerge, shifts in data distribution may occur. To analyze the drifting behavior of data instances, we partition the dataset D having n labeled data instances into set of segments $\{s_1, s_2, \ldots, s_m\}$, each of size η, where $\eta = \lfloor\sqrt{n}\rfloor$. The size of η is chosen to ensure sufficient data for reliable statistical measurements while keeping the segments small enough to detect changes in the data distribution. The segments containing malicious data instances are considered drifting segments. The subset of non-drifting segments is denoted as $S' \subseteq \{s_l, \ldots, s_m\}$, and the subset of drifting segments is denoted as $S \subseteq \{s_l, \ldots, s_m\} \setminus S'$. The learning algorithm $\mathcal{L}$ is presented with the labeled data instances. The performance of the algorithm in terms of accuracy, is represented by A. The segmentation of the dataset allows us to measure the change in data distribution and its impact on the performance of $\mathcal{L}$, denoted as Δd_l and ΔA_l, respectively, for each segment l.

To efficaciously monitor and respond to data drift, it is essential first to establish a baseline performance that reflects the expected behavior of the ML model under normal conditions. We begin by defining the baseline performance by comparing the accuracy A between S and S', thereby quantifying the impact of drift on A. Establishing this baseline helps to determine the drift detection thresholds, i.e., drift warning and drift alarm [34]. In this study, drift warning refers to when

TABLE 2. Notation table.

Notation	Description
D	Historical data: $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
D_{tr}, D_{ts}	Training and testing data
$\mathcal{L}$	Learning algorithm trained on D_{tr}
S', S	Set of non-drifting and drifting segments
$\mu_{l,j}$, $\sigma_{l,j}$	Mean and standard deviation of feature j from l-th segment
$\Delta d_{l,j}$	Magnitude of change in feature j of l-th segment
$\Delta d_{l,j}$	Cumulative magnitude of change for all features of l-th segment
A_l	Accuracy of $\mathcal{L}$ at l-th segment
θ_d	Correlation coefficient represents how the change in data distribution effects the accuracy of $\mathcal{L}$
$\sigma^2(A)$	Variance of accuracy across drifting segments
α	Drift warning threshold
β	Drift alarm threshold
W	Batch size, i.e., collection of traffic flows
M	Moving average of accuracy of last M number of batches
δ	Adjustment factor for modifying the batch size W
$\mathcal{B}_{max}$	Maximum size of the buffer
$\mathcal{R}_{rate}$	Retraining rate of $\mathcal{L}$.

the inference result degrades from the most consistent level of accuracy obtained over existing data. A drift alarm refers to an observed drop in historical accuracy due to a data distribution shift. By establishing a baseline performance for the data, we create a benchmark for future predictions.

We continuously update the thresholds based on the newly received traffic patterns. The ongoing adjustment of thresholds aims to balance drift detection accuracy and its adaptation impact. This approach enables an in-intrusion detection system to efficiently adapt to emerging patterns and maintain accuracy in evolving network behavior. The mathematical formulation for defining baseline performance and determining drift detection thresholds is provided. The necessary mathematical notations and descriptions are presented in Table 2.

We measure the change in data distribution by calculating the statistical measures, i.e., mean μ and standard deviation σ, for each feature j of l-th segment.

For non-drifting segments S':

$$\mu'_{l,j} = \frac{1}{|s_l|} \sum_{i \in s_l} x_{i,j}, \quad \forall j \in \{1, 2, \ldots, j\}, \quad \forall l \in S' \tag{1}$$

$$\sigma'_{l,j} = \sqrt{\frac{1}{|s_l|} \sum_{i \in s_l} (x_{i,j} - \mu'_{l,j})^2}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S' \tag{2}$$

For drifting segments S:

$$\mu_{l,j} = \frac{1}{|s_l|} \sum_{i \in s_l} x_{i,j}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S \tag{3}$$

$$\sigma_{l,j} = \sqrt{\frac{1}{|s_l|} \sum_{i \in s_l} (x_{i,j} - \mu_{l,j})^2}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S \tag{4}$$

Next, we calculate the difference in statistical measures between drifting and non-drifting segments:

$$\Delta\mu_{l,j} = \mu_{l,j} - \mu'_{l,j}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S, \quad \forall l \in S' \tag{5}$$

$$\Delta\sigma_{l,j} = \sigma_{l,j} - \sigma'_{l,j}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S, \quad \forall l \in S' \tag{6}$$

We calculate the magnitude of change for each feature of l-th segment to quantify the shift in data distribution. The magnitude of change for each feature j in segment l is defined as the Euclidean distance between the means and standard deviations of drifting and non-drifting segments:

$$\Delta d_{l,j} = \sqrt{(\Delta\mu_{l,j})^2 + (\Delta\sigma_{l,j})^2}, \quad \forall j \in \{1, 2, \ldots, k\}, \quad \forall l \in S \tag{7}$$

We then aggregate the magnitudes of change across all features of l-th segment to obtain the cumulative magnitude of change for the entire segment:

$$\Delta d_l = \sqrt{\sum_{j=1}^{k} (\Delta d_{l,j})^2}, \quad \forall l \in S \tag{8}$$

To measure the change in accuracy, we first calculate the baseline accuracy for non-drifting and drifting segments:

$$A'_l = \frac{1}{|s_l|} \sum_{i \in s_l} \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}, \quad \forall l \in S' \tag{9}$$

$$A_l = \frac{1}{|s_l|} \sum_{i \in s_l} \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}, \quad \forall l \in S \tag{10}$$

where *TP* indicates that the observation is positive, and the sample is predicted to be positive whereas *TN* reports that the observation is negative, and the sample is predicted to be negative. Similarly, *FP* represents that the observation is negative, but the sample is predicted to be positive. *FN* indicates that the observation is positive, but the sample is predicted to be negative.

We then calculate the change in accuracy for each drifting segment compared to non-drifting segment:

$$\Delta A_l = A_l - A'_l, \quad \forall l \in S \tag{11}$$

Next, we analyze the accuracy drops across drifting segments to determine the minimum and maximum accuracy changes observed:

$$A_{min} = \min(\Delta A_l), \quad \forall l \in S \tag{12}$$

$$A_{max} = \max(\Delta A_l), \quad \forall l \in S \tag{13}$$

To understand how changes in data distribution affect the accuracy of the model, we calculate the correlation coefficient between the magnitude of change in data distribution Δd_l and the change in accuracy ΔA_l for drifting segments S. This analysis helps in quantifying the relationship between the shift in data distribution and the observed accuracy variations, providing insight into the impact of data drift on model performance. First, we compute the average magnitude of change in data distribution and accuracy across drifting segments:

$$\overline{\Delta d} = \frac{1}{|S|} \sum_{l \in S} \Delta d_l \tag{14}$$

$$\overline{\Delta A} = \frac{1}{|S|} \sum_{l \in S} \Delta A_l \tag{15}$$

We then calculate the correlation coefficient θ_d that represents how the change in data distribution affects the obtained accuracy and is used to verify the drift occurrence before triggering the model updates:

$$\theta_d = \frac{\sum_{l \in S}(\Delta d_l - \overline{\Delta d})(\Delta A_l - \overline{\Delta A})}{\sqrt{\sum_{l \in S}(\Delta d_l - \overline{\Delta d})^2}\sqrt{\sum_{l \in S}(\Delta A_l - \overline{\Delta A})^2}} \tag{16}$$

Finally, we calculate the variance of the accuracy $\sigma^2(A)$ across drifting segments:

$$\sigma^2(A) = \frac{1}{|S|} \sum_{l \in S} (\Delta A_l - \overline{\Delta A})^2 \tag{17}$$

To determine the drift warning α and drift alarm β thresholds, we define them based on baseline performance, where

$$\alpha = A_{max} - 1 \tag{18}$$

$$\beta = \sigma^2(A) \tag{19}$$

The drift warning threshold α is set as $A_{\max} - 1$ to signal a one-point drop from the maximum observed accuracy, while the drift alarm threshold β is set as the variance $\sigma^2(A)$ to manage the tradeoff between detecting significant drifts promptly and smoothing short-term fluctuations in accuracy across drifting segments. We keep updating β on unseen data, that is, a collection of traffic flows W:

$$\beta = \frac{1}{M} \sum_{p=q-M+1}^{q} A_p(W) \tag{20}$$

where:

$$A_{min} \leq \beta < \alpha < A_{max} \tag{21}$$

The continuous updates to the parameter β are based on the average accuracy of the last M batches, i.e., collections of traffic flows, each denoted as W. Let $M = 4$, where M represents the number of batches used to calculate the moving average. The value of M is chosen as a typical tradeoff between detecting drift responsively and smoothing short-term fluctuations in accuracy. This balance ensures that the system can react promptly while avoiding over-sensitivity to minor variations. Let q be the current index of W, and the index p iterates over the previous M flow collections, from $q-M+1$ to q. The denominator M normalizes the sum of the M collections. Additionally, we bound the parameters α and β using the minimum and maximum accuracy obtained from the baseline performance to prevent the system from continuously being in a drifting state.

The parameter W is dynamically adjusted using an adjustment factor δ to enhance adaptability to unseen traffic patterns:

$$\delta = (1.0 - A_q) \times W \tag{22}$$

$$W = \begin{cases} W + \delta & \text{if accuracy is increasing} \\ W - \delta & \text{if accuracy is decreasing} \\ W & \text{otherwise} \end{cases} \tag{23}$$

The adjustment factor δ is introduced to modify W based on the current accuracy level. The degree of adjustment depends on the disparity between the current accuracy and the desired level. A larger increment or decrement to W occurs when the accuracy significantly deviates from the desirable level. Consequently, the monitoring parameter W becomes more responsive to updates during severe accuracy drops and adjusts more conservatively for minor accuracy deviations.

2) DATA DRIVEN APPROACH

The complete drift detection and adaptation processes are outlined in Algorithms 1 and 2, respectively. The detection algorithm uses historical and unseen data samples as input, along with the initially determined parameters and thresholds. A *STATE* variable initialized to *zero* at line 3 signifies the *normal, warning, and drifting* conditions of the system. The subsequent lines initialize the buffer and the adjustment factor for W.

Algorithm 1 iterates over unseen data D_{ts} from lines 6 to 31, continually adjusting the thresholds (β; W, which is initially set to the determined segment size η; and $\mathcal{B}_{max}$) to strike a balance between drift detection accuracy and its adaptation impact. For each W, which represents the monitoring flow count, the obtained accuracy is assigned to variable A_q at line 7. The state of the system is reflected by the current accuracy A_q. The first condition checks if the system is non-drifting (i.e., when the *STATE* is 0) but A_q has dropped from the determined α, then the *STATE* becomes *one* that indicates a drift warning. When there is a drift warning, buffer $\mathcal{B}$ begins to keep logs (i.e., in-band extracted features information) of incoming flows until the maximum buffer size is reached (see lines 10-17). We define $\mathcal{B}_{max}$ as the stopping condition for buffering. Meanwhile, if the accuracy increases back to the threshold α, the algorithm considers the drift warning to be a false alarm, resets the buffer at line 16, and reinitializes the *STATE* to *zero*. If A_q further degrades to the drift alarm threshold β or the buffer size reaches $\mathcal{B}_{max}$, the system transitions to the drift alarm state (see line 14), and the drift verification and adaptation process starts from line 18.

Algorithm 1 Drift Detection

Input: D_{tr}, D_{ts}, $\mathcal{L}$, α, β, W, $\mathcal{B}_{max}$, M, θ_d, $\mathcal{U}$, X_{SHAP}, FS_{KBest}
$\mathcal{U}$: Object of unsupervised learning algorithm,
X_{SHAP}: Object of feature significance calculator,
FS_{KBest}: Object of feature selection method called 'top K best'
Output: A[], $\mathcal{R}_{rate}$, decisionRules

1 $\mathcal{R}_{count} = 0$, $\mathcal{R}_{rate} = 0$, $A = []$;
2 shapValues = [], decisionRules = [];
3 STATE $\leftarrow 0$; // current state of drift detection process
4 $\mathcal{B} \leftarrow \theta$; // buffer
5 $\delta \leftarrow \theta$; // adjustment factor for W
6 **while** W_q *in* D_{ts} **do**
7 $\quad A_q \leftarrow$ accuracy(W_q);
8 $\quad$ **if** *STATE == 0 AND* $A_q < \alpha$ **then**
9 $\quad\quad$ STATE $\leftarrow 1$; // indicates a drift warning
10 $\quad$ **if** *STATE == 1 AND* $\mathcal{B} < \mathcal{B}_{max}$ **then**
11 $\quad\quad$ // $\mathcal{B}$ indicates current size of buffer
12 $\quad\quad \mathcal{B} \leftarrow W_q$;
13 $\quad\quad$ **if** $A_q < \beta$ *OR* $\mathcal{B} == \mathcal{B}_{max}$ **then**
14 $\quad\quad\quad$ STATE $\leftarrow 2$; // indicates a drift alarm
15 $\quad\quad$ **if** $A_q > \alpha$ **then**
16 $\quad\quad\quad \mathcal{B} \leftarrow \theta$;
17 $\quad\quad\quad$ STATE $\leftarrow 0$;
18 $\quad$ **if** *STATE == 2* **then**
19 $\quad\quad \Delta d_q = \sqrt{\sum_{j=1}^{k} (\Delta d_{l,j})^2}$;
20 $\quad\quad$ **if** $\Delta d_q > \theta_d$ **then**
21 $\quad\quad\quad$ STATE $\leftarrow 0$ // indicates a non-drifting state
22 $\quad\quad\quad$ *decisionRules* = DriftAdaptation(D_{tr}, $\mathcal{B}$, $\mathcal{L}$, $\mathcal{U}$, X_{SHAP}, FS_{KBest});
23 $\quad\quad\quad \mathcal{R}_{count} = \mathcal{R}_{count} + 1$;
$\quad\quad\quad$ // model retraining counter
24 $\quad$ *A.append*(A_q);
25 $\quad \beta = \frac{1}{M} \sum_{p=q-M+1}^{q} A_j$;
26 $\quad \delta = |1.00 - A_q| \cdot W_q$;
27 $\quad$ **if** $A_q < A_{q-1}$ **then**
28 $\quad\quad W_q = W_q - \delta$;
29 $\quad$ **if** $A_q > A_{q-1}$ **then**
30 $\quad\quad W_q = W_q + \delta$;
31 $\quad \mathcal{B}_{max} = M \cdot W_q$;
32 $\mathcal{R}_{rate} = \mathcal{R}_{count}/k$; // calculate model retraining rate
33 **return** *A[],* $\mathcal{R}_{rate}$*, decisionRules []*

During the drift alarm, the system calculates the change in data distribution for all features $j \in \{1, 2, \ldots, k\}$ of the buffered instances. A predetermined threshold θd is used to detect changes in the data distribution. The threshold helps verify whether the distribution of buffered instances Δd_q has changed enough to indicate a drift. If this condition is true, the drift adaptation process begins by calling an adaptation algorithm at line 22. The adaptation algorithm takes the historical D_{tr} and buffered instances $\mathcal{B}$, along with the unsupervised learning algorithm and explainability methods as inputs, and returns updated decision rules for online inference in the data plane. Each time the adaptation algorithm is called, the model retraining count variable R_{count}, which represents the retraining rate, is incremented by one.

Algorithm 2 Drift Adaptation

Input: D_{tr}, $\mathcal{B}$, $\mathcal{L}$, $\mathcal{U}$, X_{SHAP}, FS_{KBest}
Output: decisionRules []

1 $\mathcal{B} = \mathcal{U}(\mathcal{B})$ // Label buffered data using unsupervised learning
2 $\mathcal{L}$. fit$(D_{\text{tr}} + \mathcal{B})$;
// Retrain model with training and labeled buffer data
3 currentShap = $\mathcal{L}$.featureImportance(X_{SHAP});
4 // Compute SHAP values for feature importance
5 shapValues.append(currentShap);
6 currentTopKFeatures $= FS_{\text{KBest}}(shapValues)$;
7 // Select top K features
8 **if** *currentTopKFeatures* $\neq$ *prevTopKFeatures* **then**
9 $\quad$ Trigger: 'update data plane
10 $\quad$ monitoring sketch for top K features';
11 prevTopKFeatures = currentTopKFeatures;
12 decisionRules [] = $\mathcal{L}$.getRules(); // Get updated decision rules
13 **return** *decisionRules []*

During drift adaptation, Algorithm 2 first applies an unsupervised learning algorithm to $\mathcal{B}$ to label instances for the model retraining. Next, the model is retrained on $\mathcal{B}$ and D_{tr} at line 2. From lines 3-7, explainability methods that focus on identifying important features are applied to the predictions of ML model to extract the top K features that contribute the most to the predictions of the classifier. The record of the top K features is maintained, triggering the need for sketch updates in the data plane if the relevance of the features changes over time. Note that P4-enabled switches only support runtime updates of *MATs* but lack full reprogrammability at runtime, requiring a switch reboot for modifications [35]. Therefore, updating the sketch for the top K features necessitates rebooting the switch, as it is an architectural constraint. Finally, the updated decision rules are derived at line 11 and returned as the output of the algorithm.

Because we opted for a data-driven approach, we continually adjusted the drift detection thresholds based on current classification accuracy. Algorithm 1 adjusts the thresholds at lines 25-31. A moving average parameter M is defined to reflect the current classification performance to β; hence, β is iteratively adjusted based on the average accuracy of the last M flow collections (see line 25). Additionally, instead of using a constant W, we adjust W to reflect the dynamics of accuracy. For instance, when drift occurs, W becomes more aggressive and frequently triggers retraining by decreasing size and vice versa. A δ variable is used to adjust W, reflecting the change in the current accuracy A_q. Hence, the effect of δ on W is based on the current performance of ML model and size of W. The following conditions use δ to increment or decrement into W. Finally, the model retraining rate $\mathcal{R}_{rate}$ is calculated by dividing the number of retraining events i of W by R_{count}.

TABLE 3. Dataset summary.

CICIDS2017		UNSW-NB15	
Label	Flows	Label	Flows
BENIGN	19101	BENIGN	40550
Hulk	34587	Exploits	4033
GoldenEye	1540	Fuzzers	2914
Slowhttptest	843	Generic	593
Slowloris	824	DoS	569

IV. VALIDATION METHODOLOGY

This section provides an overview of the datasets used for validation, explains the rationale behind feature selection and the choice of ML algorithm, the unsupervised methods for labeling, and presents the experimental setup in Subsections V-A, V-B, V-C, and V-D.

A. DESCRIPTION OF DATASETS

We use two network intrusion detection datasets to evaluate the proposed method for mitigating evolving volumetric attacks. The first dataset CICIDS2017, [19] provided by the Canadian Institute of Cybersecurity (CIC), which includes the most updated cyberattack scenarios. As different types of attacks are launched at various times to create the dataset, the attack patterns change over time, causing multiple drifts. The second dataset, UNSW-NB15 [20], is developed at the University of New South Wales and captures a wide range of attack types. Like CICIDS2017, UNSW-NB15 spans different periods, reflecting the dynamic landscape of cyber threats and introducing variations in attack patterns.

The datasets consist of traffic patterns gathered over several days. However, we focus on specific portions to facilitate the evaluation process. In the case of the CICIDS2017 dataset, we select the traffic portion generated specifically on Wednesday, July 5, 2017. For UNSW-NB15, we utilize a 16-hour simulated period on January 22, 2015. It is important to note that using all data samples for model development is often impractical and unnecessary. Therefore, an effective data-sampling method is not just beneficial, but essential for selecting highly representative data. A well-chosen sample can significantly reduce computational complexity, enhance model performance, and prevent overfitting by focusing on the most relevant data points. In our case, we employ the k-means cluster sampling method, a technique that groups data samples based on similarity and selects a proportion of samples from each cluster. Given the substantial size of the datasets, this method allows us to choose 10% of the original data from each dataset for evaluating the proposed framework. Importantly, the k-means cluster sampling method generates a highly representative and high-quality subset by discarding redundant data points, distinguishing it from other sampling methods.

Following the implementation of the k-means clustering sampling method, two representative subsets are obtained: the CICIDS2017 subset, which contains 56,895 instances, and the UNSW-NB15 subset, which includes 48,659 instances. The subsets serve as the basis for evaluating the proposed method. The datasets are listed in Table 3. Notably, both datasets exhibit considerable imbalance, with normal/abnormal ratios of 34%/66% and 81%/19%, respectively. This ratio reflects the proportion of benign (normal) traffic to malicious (abnormal) traffic within the datasets. A higher percentage of abnormal traffic in CICIDS2017 (66%) and normal traffic in UNSW-NB15 (81%) demonstrates the inherent variability in the datasets, which is crucial for evaluating the robustness of the ML models on imbalanced datasets.

Since anomaly detection systems aim to distinguish cyberattacks from normal states, dataset instances are treated as binary, with two labels: *benign* or *malicious*. For model evaluation, we employ both hold-out and prequential validations. In the hold-out evaluation, the initial model training uses the first 10% of the data, whereas the remaining 90% is used for testing. In prequential validation, also known as test-and-train validation, batches of instances (i.e., segments) in the inference test set are first used to test the learning model, and subsequently employed for model retraining and updating.

TABLE 4. Features definition.

Feature Type		Description
Per-packet	-	Packet size, protocol, etc.
Flow-level	Aggregate Summary	F = aggr(a, c, d) max, min and mean, etc.

B. FEATURES AND MODEL SELECTION

Given the constraints of the PDP, extracting a minimal set of features from network packets is crucial without requiring complex operations. Therefore, a feature extraction and selection module is employed. Table 4 lists the selected features for both datasets. The five-tuple attributes (source IP, destination IP, source port, destination port, and protocol) serve as primary flow identifiers since they are available in packet headers and can be processed at line rate within P4-enabled switches. While volumetric attacks often exhibit distributed characteristics across multiple IPs and ports, they still induce detectable traffic anomalies, such as sudden spikes in flow volume, changes in packet size distributions, and irregular protocol usage patterns. To capture such variations, we extend the five-tuple information by incorporating flow-level statistical summaries, including the minimum, maximum, and mean packet size per flow. These additional features allow the system to detect shifts in traffic characteristics.

Since the relevance of features may change due to concept drift, we use explainability techniques after each model retraining to assess and retain only the most relevant features. The SHAPley value-based explainability method [36] identifies the top K features that contribute most to classification accuracy, ensuring that important flow characteristics remain included in the decision-making process. This explainability-driven feature selection approach enables the data plane to continuously adapt its feature set while maintaining a lightweight yet effective classification mechanism.

Although the feature such as source ports is often randomized, feature importance analysis (see Fig. 6) shows that they exhibit measurable relevance in certain traffic segments, particularly in attack patterns. This suggests that, in specific cases, automated attack tools or misconfigurations lead to non-random port usage.

The choice of an ML model depends on its deployability within a PDP. Although various supervised learning approaches exist for traffic characterization, not all are compatible with the implementation in P4 [14]. We aim to seamlessly integrate the ML model into the data plane, which requires alignment with the available operations in P4. Considering the current primitives in P4 [4], a *Decision Tree Classifier (DTC)* emerges as the optimal choice for this task [37], [38]. The classification process of the DTC, involving comparison operations for element x, aligns seamlessly with the *MATs* of the PDP.

C. UNSUPERVISED LABELING

To autonomously classify network traffic, we employ k-Means clustering [39] and Isolation Forest (iForest) [40] as unsupervised learning techniques to identify malicious and benign instances from unseen traffic patterns. k-Means partitions traffic based on similarity, assuming that benign and attack traffic exhibit distinct clustering characteristics. iForest, on the other hand, detects anomalies based on the isolation of sparse patterns typically associated with attack traffic. The choice of unsupervised learning is motivated by the need to operate without labeled data, ensuring adaptability to evolving attack patterns. We evaluate both models across different datasets and analyze their efficacy in correctly identifying labels before the retraining processes.

D. EXPERIMENTAL SETUP

We structured our experiments in three steps. First, we detail our simulation setup in Subsection IV-C1. Next, we explain the ML model training and deployment in the data plane in Subsection IV-C2. Finally, we describe the performance measures for in-network attack detection in Subsection IV-C3.

1) SIMULATION SETUP

The logical components of the simulation setup are shown in Fig. 3. The *measurement component* (on the left) generates, collects, and analyzes the network traffic. The *data plane component* (on the right) serves as the focal point of evaluation, implemented in P4 and compiled with the behavioral model version 2 (BMv2) [41]. The structure of the simulation setup is as follows:

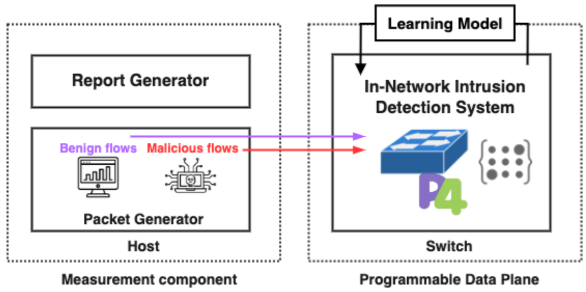

FIGURE 3. Simulatiing data plane.

- The host uses the Python library *dpkt*[2] to send packets and receives them back with classification and timestamps. This component reports the classification results.
- The switch contains a traffic classification algorithm responsible for detecting anomalies and either forwarding or discarding packets.
- The learning module atop the data plane manages the ML model training and interpretation process on the provided data and is implemented in the control counterpart of the network.

2) MODEL TRAINING AND DEPLOYMENT

In order to seamlessly integrate ML model with PDP, we require tools that support both model training and the interpretation of learned rules for real-time inference. We use the *scikit-learn*[3] implementation in Python for the ML model training. The learned rules of the *DTC* classifier are extracted and subsequently translated into *MATs* as *MA* rules using the P4Runtime API [32]. Each *MAT* corresponds to a single feature, with the total number of tables equal to the number of input features plus a class table.

Before applying the *MATs*, the P4 code extracts the port numbers and packet size from sequential packets of each flow and stores them in the memory registers in *SRAM*. Once the threshold is reached (i.e., the minimum number of packets required for a classification decision), the switch generates a summary of the input feature values. The extracted feature values are then processed through *MATs* to determine the traffic class.

3) PERFORMANCE MEASURES

The evaluation aimed to efficaciously address the detection and adaptation of concept drift with minimal disturbance to normal network traffic. As concept drift represents emerging attacks in our case, our goal is to enable a continuous learning approach to quickly detect and mitigate malicious traffic.

Quick detection and adaptation of concept drifts are desirable. However, performing such operations for every instance is infeasible in a pragmatic framework. ADaptive WINdowing (ADWIN) and Drift Detection Method (DDM) are

[2]https://pypi.org/project/dpkt/
[3]https://scikit-learn.org/

two common drift detection techniques [42]. ADWIN is a distribution-based method that effectively addresses gradual drift by increasing window size. On the other hand, DDM is a model performance-based method that defines warning and drift level thresholds to monitor the model error rate and standard deviation change for detection. Although the DDM can identify sudden drifts, its response time is often slow for gradual drifts. Motivated by the effectiveness of ADWIN, we implement an adaptive windowing-enabled DDM to monitor the accuracy of the collection of flows with a dynamic window size and then verify the drift by measuring the change in data distribution. Hence, we opted to use an ADWIN-enabled DDM method to maintain the efficacy of drift detection and adaptation in programmable networks by preserving detection accuracy and controlling adaptation impact.

In our proposed framework, the ADWIN-enabled DDM continuously monitors the model predictions and adjusts the window size W (i.e., the batch of flows to be monitored) based on the current inference result. For instance, when the model undergoes drifts, W becomes smaller to adapt to new patterns quickly, while in the absence of drift, the window becomes larger to reduce the impact of adaptation.

Several performance metrics are used to evaluate the framework. One set of measures relates to drift detection accuracy, with two key metrics: True Positive Rate (TPR) and True Negative Rate [15]. TPR is the ratio of correctly classified positive samples to the total number of positive samples, while TNR is the ratio of correctly classified negative samples to the total number of negative samples. Hence, TPR and TNR represent the percentage of malicious flows that were correctly predicted and the percentage of benign flows that were correctly predicted, respectively.

$$TPR = \frac{TP}{TP + FN}$$
$$TNR = \frac{TN}{TN + FP}$$

The other set of metrics concerns the impact of drift adaptation, which is represented as packet loss caused by model retraining. Because the ML model is implemented in the form of *MA* rules in the *MATs* of the PDP, retraining causes model remapping to the existing *MATs*. In other words, when the control plane generates a fresh set of rules, these rules are written to the data plane to form new inference thresholds and complete the updating process. Such an update disturbs normal traffic processing of the network. Therefore, the framework should maintain the efficacy of concept drift detection with a minimal adaptation impact. Hence, we defined the model retraining and packet loss rates as metrics to be minimized, with the former directly reflecting the latter.

Both sets of metrics are directly reflected by the monitoring window size W. A smaller W may result in quick drift detection but will lead to more frequent updates in the network. Conversely, a larger W will reduce the adaptation impact but might not efficaciously address the drift. Hence, maintaining the efficacy of drift detection and minimizing the effects of adaptation are equally important.

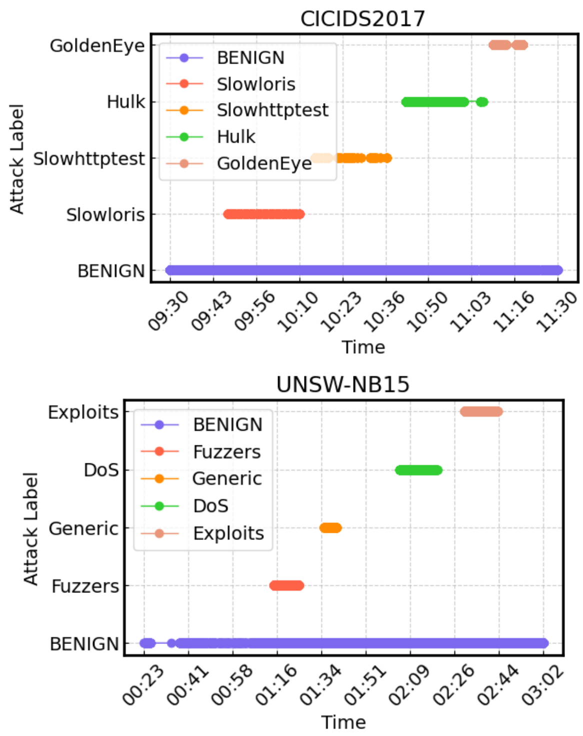

FIGURE 4. Attacking traffic timelines.

V. EXPERIMENTAL RESULTS

The experimental results are presented in three parts. First, Subsection V-A validates the existence of drifts and their implications. Next, Subsection V-B evaluates the efficacy of the proposed data-driven method on unseen data. Finally, Subsection V-C presents the system overhead, while Subsection V-D discusses the limitations of the current work and potential research directions.

A. VALIDATING DRIFTS IN THE DATASETS

The datasets selected for the analysis include evolving attack traffic patterns generated at different time intervals. We visualized the timelines of the attacking traffic instances from both datasets. Fig. 4 shows the emergence and evolution of attack patterns over time for CICIDS2017 at the top and UNSW-NB15 at the bottom. The *x-axis* represents the timelines, and the *y-axis* denotes the attack type. Notably, benign traffic is consistently present, whereas various attacks are introduced at distinct points in time.

1) CHANGE IN DATA DISTRIBUTION

We further examined variations in the traffic patterns that deviate from the norm. Attackers attempt to create traffic with diverse characteristics, such as using different ports and introducing randomness in packet sizes or inter-arrival times. The variations in traffic characteristics lead to changes in

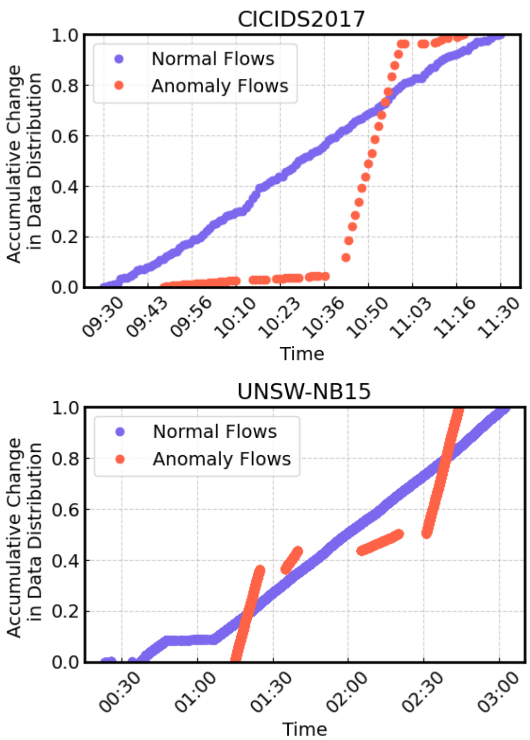

FIGURE 5. Effect of emerging traffic patterns on change in data distribution.

the distribution of malicious traffic and the traffic patterns become unknown to the existing ML model.

To better understand these deviations, we calculate the cumulative change for all features of benign and malicious traffic flows to investigate how the data distribution of anomalous flows deviates from normal flows. The sub-figures in Fig. 5 show the cumulative change in the data distribution for the chosen datasets, CICIDS2017 (on top) and UNSW-NB15 (at the bottom). The *x-axis* represents the timeline, and *y-axis* shows the cumulative change within a range of 0 to 1. Notably, the distribution of the benign traffic exhibits a linear change while maintaining consistent characteristics. In contrast, the distribution of malicious traffic differs, occasionally showing an exponential increase and, at other times, a slower progression.

2) CHANGE IN FEATURES RELEVANCE

We extended our analysis to investigate how emerging attacks impact feature relevance in intrusion detection. To capture these effects, we divided the dataset into ten sequential segments, each representing a distinct traffic pattern, and assessed feature relevance changes over time.

Fig. 6 illustrates how feature importance evolves dynamically across these segments. In CICIDS2017, packet-level features initially play a dominant role in classification. However, as new traffic patterns emerge–potentially due to attack bursts or evolving normal behavior—other features, such as flow-based statistics, gain importance. This shift suggests that attacks in CICIDS2017 exhibit diverse characteristics, necessitating continuous adaptation in feature selection.

In contrast, UNSW-NB15 demonstrates a more stable feature relevance pattern. The maximum packet size, a flow-level feature, consistently remains the most influential across all segments, while the relevance of other features fluctuates only slightly. This indicates that the types of attacks in UNSW-NB15 are more uniform over time, leading to a less dynamic feature importance profile.

These observations highlight the necessity of adaptive feature selection. In datasets like CICIDS2017, where attack characteristics evolve significantly, relying on static feature sets may degrade detection performance. By contrast, in datasets with more stable attack distributions like UNSW-NB15, static features may remain effective over extended periods.

From an implementation perspective, updating the sketch to maintain an up-to-date feature matrix in the PDP requires rebooting the device. To respect this architectural constraint, our evaluation retains the feature matrix across all segments while still capturing the shifts in feature relevance for attack indication. This ensures a practical balance between adaptability and deployment feasibility in real-time network security applications.

3) CHANGE IN MODEL ACCURACY

Fluctuations in feature relevance and their classification decision boundaries, caused by changes in data distribution, may lead to incorrect predictions by the ML model, thereby degrading its performance over time.

The performance degradation of the ML model is evaluated by varying the size of the training set from 15% to 75%. The decrease in accuracy across different training sets is shown in Fig. 7. We assess the impact of evolving traffic patterns and examine the effect of model retraining over subsequent data segments.

As attackers employ increasingly sophisticated methods such as generating adversarial traffic to mimic legitimate patterns and evade intrusion detection systems, we introduce adversarial data samples to evaluate the retraining effect in such scenarios. The results are depicted in Fig. 7, where the upper row corresponds to existing traffic instances from the selected datasets, and the lower row represents adversarial traffic. The red dots indicate the points where the test set commences.

For normal traffic patterns (upper row in Fig. 7), accuracy initially decreases when encountering previously unobserved traffic segments but gradually improves with subsequent retraining. This demonstrates that adaptation over time mitigates the effects of evolving traffic patterns by enabling the model to learn new variations in both normal and attack behaviors. However, for adversarial traffic patterns (lower row in Fig. 7), accuracy degradation is more pronounced. Even with retraining over subsequent segments, the ML model struggles to regain its previous performance, suggesting that adversarially crafted traffic significantly disrupts

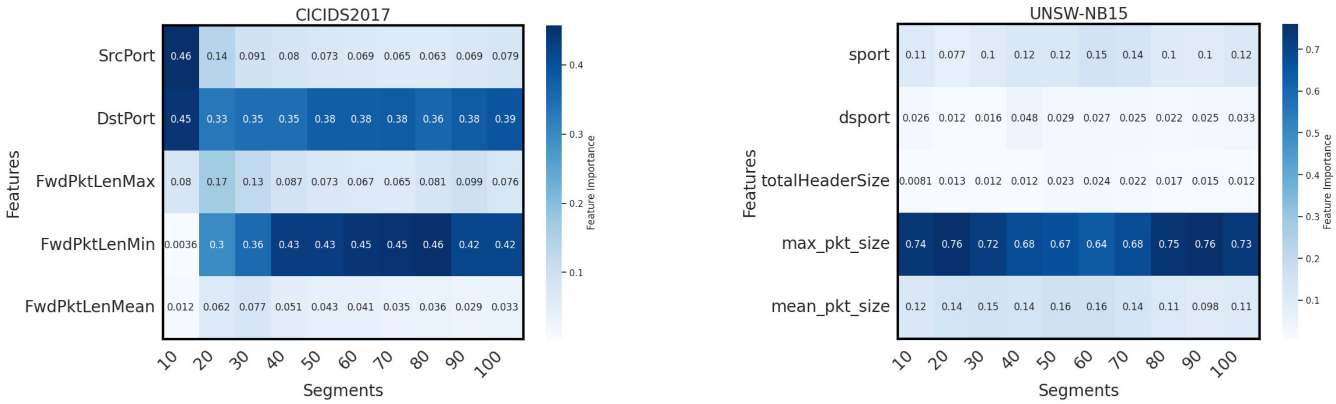

FIGURE 6. **Effect of emerging traffic patterns on change in features relevance.**

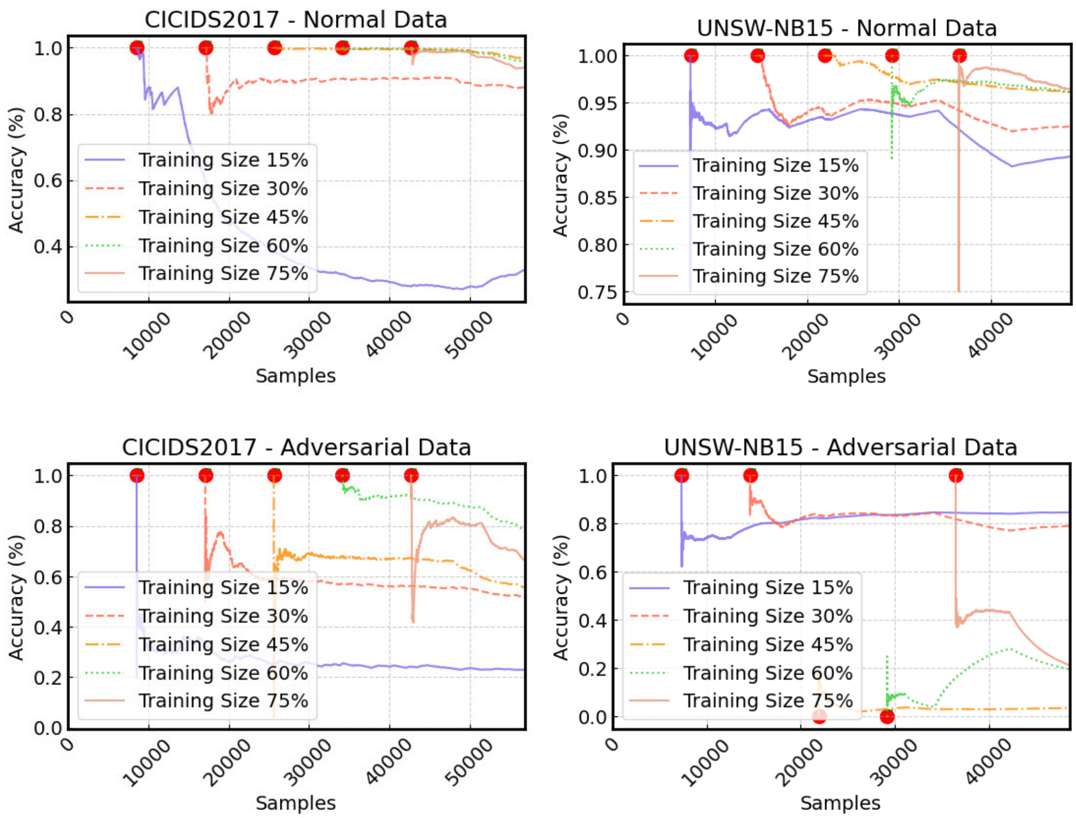

FIGURE 7. **Evaluating accuracy across training sets under normal and adversarial conditions.**

classification consistency. This indicates that adversarial traffic introduces greater divergence from learned representations, presenting a challenge for traditional ML models to adapt effectively.

4) PERFORMANCE OF UNSUPERVISED LEARNING METHODS

To evaluate the effectiveness of unsupervised learning methods i.e., k-Means and iForest, we assessed their classification performance on the chosen datasets: CICIDS2017 and UNSW-NB15. The evaluation measures the average classification accuracy (%) across multiple retraining iterations, as illustrated in Fig. 8.

The findings indicate that model performance exhibits variability across different datasets. On the CICIDS2017 dataset, the k-Means algorithm initially demonstrates moderate accuracy, which improves with retraining, ultimately stabilizing above 90%. Conversely, the iForest algorithm experiences a significant decline in performance after several retraining cycles and does not recover. In the case of

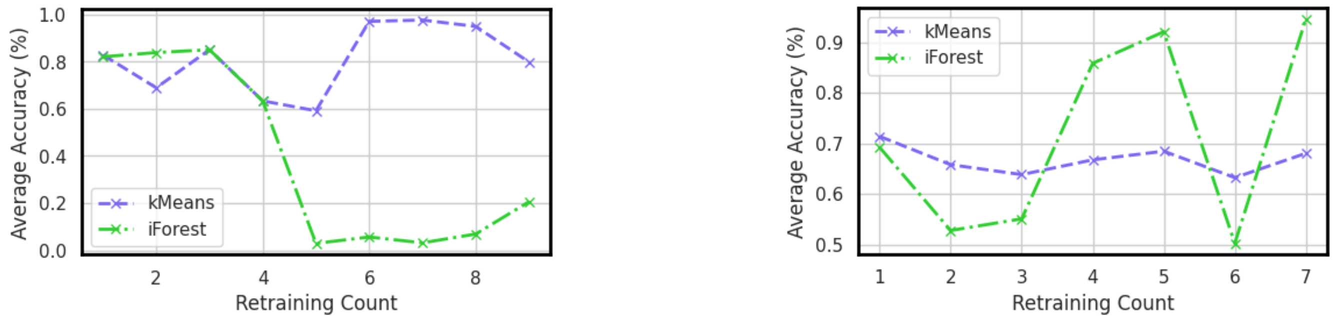

FIGURE 8. **Classification accuracy of unsupervised learning methods for CICIDS2017 (on left) a d UNSW-NB15 (on right).**

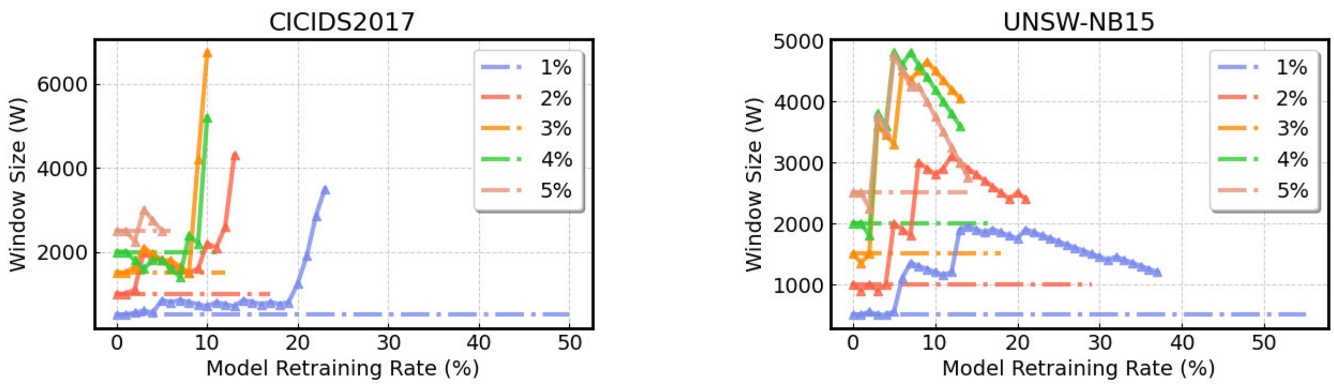

FIGURE 9. **Effect of window size (W) on ML model retraining rate.**

the UNSW-NB15 dataset, iForest performs markedly better, achieving over 90% accuracy following multiple retraining iterations, whereas k-Means remains stable but is confined to the 65–75% accuracy range. These discrepancies arise from the inherent assumptions of the algorithms and their interaction with the characteristics of the datasets. The k-Means algorithm performs effectively on CICIDS2017 due to the dataset's clustered and periodic attack patterns, which align with its centroid-based partitioning approach. Its dependence on compact clusters facilitates the detection of recurring attack behaviors. In contrast, iForest excels on UNSW-NB15, which features sparse, diverse, and irregular attack distributions. Its isolation-based mechanism does not rely on predefined cluster shapes, rendering it well-suited for identifying scattered or low-frequency anomalies. This comparison highlights the critical importance of aligning unsupervised detection models with the structural properties of network traffic data.

B. EVALUATING THE EFFICACY OF DATA DRIVEN APPROACH

In this subsection, we evaluate the efficacy of the proposed data-driven method by demonstrating its ability to identify and mitigate emerging malicious traffic flow. To achieve this, we adopt drift detection thresholds based on the current performance of ML model. We then analyze the impact of varying monitoring window sizes on the efficacy of drift detection and its adaptation impact. Our results show that the proposed approach efficaciously detects and mitigates evolving malicious traffic.

1) IMPACT OF VARYING WINDOW SIZE

The term *window size* (W) refers to a batch of traffic flows representing the monitoring frequency in our pragmatic framework. Monitoring every instance is impractical and can result in a significant network overhead. A smaller W makes the system more aggressive in monitoring model performance, whereas a larger W reduces the monitoring frequency.

The efficacy of our design lies in using an adaptive windowing-enabled drift detection method, the ADWIN-DDM. We highlight the potential of ADWIN-DDM compared to DDM with static W. The subfigures in Fig. 9 depict the effect of W on the model retraining rate for the chosen datasets. The *x-axis* represents the size of W, and the *y-axis* represents the model retraining rate. The simulation is performed using the selected portion of the datasets, which is divided into 10% for training and 90% for testing. The initially trained ML model incrementally adapts incoming flow instances in batches of size W. The dashed-dotted lines represent the DDM with W, whereas the solid lines with arrow markers represent the DDM with dynamic W. Various

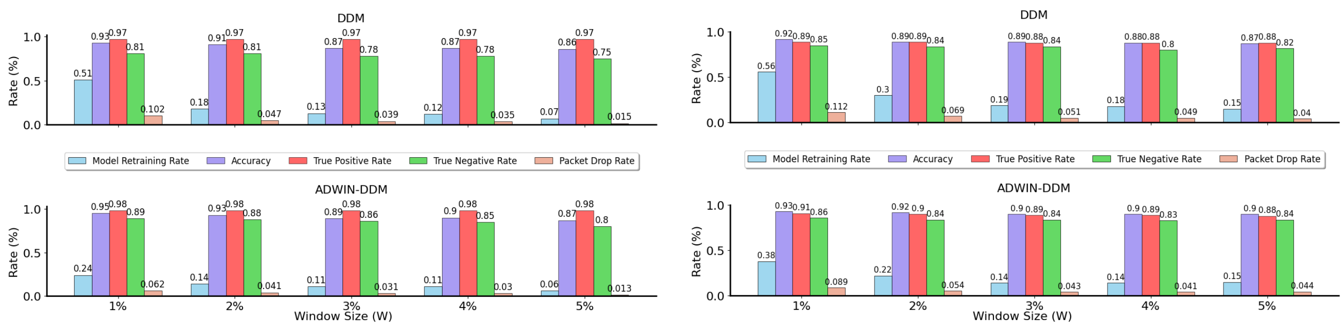

FIGURE 10. Performance measures: CICIDS2017 (left) and UNSW-NB15 (right).

starting sizes of W (ranging from 1% to 5% of the total flow instances) are represented by different colors proportionate to the chosen datasets.

The results show that the impact of the DDM with static W is directly proportional to the model retraining rate for both datasets. In contrast, DDM with an adaptive window size exhibits a different behavior. The solid lines demonstrate the dynamic adjustments of W based on the model performance. In the case of static W, a smaller window leads to more frequent adaptation, whereas a larger W slows adaptation. However, the dynamic W is adjusted based on the nature of the data and model performance. When the model performance decreases, W becomes more aggressive to address drift quickly. When the model performance improves, W becomes larger to avoid unnecessary updates and reduce or eliminate the negative impact of adaptation. The variation generally shows similar trends regardless of the starting size, indicating an adjustment to W based on the nature of the data and model performance.

Overall, the adjustment of W is found to be more effective in capturing the characteristics of the unseen data from both datasets. In the case of the CICIDS2017 dataset, the size of W initially fluctuates but then increases considerably as the accuracy of the model improves, leading to a decrease in the retraining rate. In contrast, the unseen traffic patterns in the UNSW-NB15 dataset are largely unknown to the trained ML model, resulting in significant fluctuations in the size of W. Therefore, the adjustment of W is made based on the nature of the unseen data throughout the simulation, resulting in a better capture of unseen patterns with minimal adaptation impact.

Fig. 10 provides a more precise view of the effect of W on the drift detection accuracy and its adaptation impact for both datasets: CICIDS2017 on the left and UNSW-NB15 on the right. The grouped bars on the *x-axis* represent the performance metrics, and the *y-axis* represents the results in percentages. The first metric, the model retraining rate, is proportional to the initial window sizes in both cases (DDM with constant and dynamic W). The retraining rate directly influences the detection accuracy, as more frequent model adaptation of new patterns allows quicker identification of emerging traffic patterns. However, this comes at a cost represented by the packet drop rate, which is directly affected by the model retraining rate. Hence, there is a trade-off between obtaining the classification accuracy and minimizing the model retraining impact, which is indicated in both cases for both datasets.

The results unequivocally demonstrate the efficacy of using a dynamically adjusted W over a static W. For CICIDS2017, the model retraining rate is significantly reduced, and the drift detection accuracy increases slightly when W started with 1% of the given instances. The subsequent sizes of W exhibit similar trends, with a trade-off between the accuracy and packet drop rate. The results for UNSW-NB15 on the right show similar trends over the complete simulation but with different rates due to the adjustment of W depending on the unseen data. Overall, the performance results are directly reflected by W in all cases for both datasets, indicating a negative correlation between W and model retraining. The ML model retraining has a positive impact on the efficacy of drift detection but a negative impact on drift adaptation. Therefore, the data-driven approach allows for the dynamic adjustment of W based on the nature of incoming data, which better manages the trade-off by maintaining accuracy while minimizing the impact of adaptation.

2) MITIGATING EVOLVING MALICIOUS TRAFFIC

The efficacy of the proposed approach in mitigating evolving malicious traffic is illustrated in Fig. 11. The 1^{st} *y-axis* on the left represents the average accuracy, and the $2^{n}d$ *y-axis* on the right depicts the number of predictions for each class, that is, the count of benign and malicious flows. The simulation runs over the chosen portion of datasets, with 10% of the data for training and 90% for testing, initializes W at 1% of the total data. The subfigures display the results from the selected datasets using three cases distinguished by color (red, cyan, and blue): no adaptation, i.e., baseline; adaptation using static W (i..e, DDM); and adaptation using dynamic W (i.e., ADWIN-DDM). The solid lines represent the average accuracy, the dashed-dotted line represents the number of positive predictions (malicious flows), and the single dashed

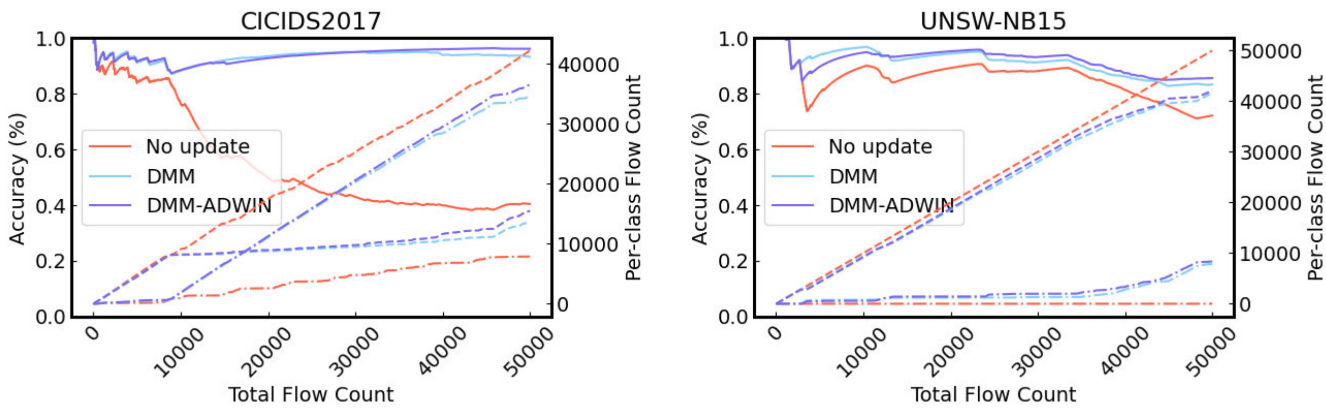

FIGURE 11. **Data plane results.**

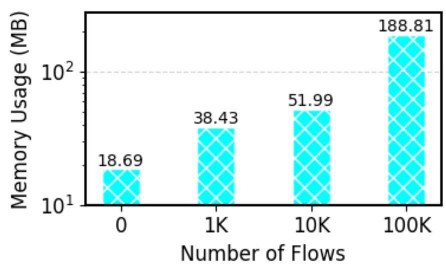

FIGURE 12. **Impact of concurrent flows on SRAM usage.**

line indicates the number of negative predictions (benign flows).

The subfigure on the left for CICIDS2017 reveals that the average accuracy continuously decreases to 40% when the model was not retrained. Such a decrease is evident in the prediction counts, where malicious flows remain above 10,000, whereas the count of benign flows exceeds 40,000. By contrast, both model adaptation cases demonstrate how the model maintains accuracy by correctly identifying malicious and benign flows. In both cases, the model accurately identifies anomalies when new patterns emerge. The subfigure on the right presents the results for the UNSW-NB15. The average accuracy of the model without adaptation significantly decreases. In the adaptation cases, the accuracy is maintained by correctly mapping instances to their respective classes. However, there is a variation in the obtained accuracy attributed to the dissimilarities of the new patterns with the existing ones. Additionally, from both datasets, the precedence of ADWIN-DDM over DDM is indicated in obtaining better accuracy by more precisely identifying malicious traffic, facilitated by its dynamic adjustment to W based on model performance. In other words, ADWIN-DMM achieves better classification accuracy by efficaciously capturing and adapting the evolving shifts in malicious traffic patterns.

C. SYSTEM OVERHEAD

This subsection quantifies SRAM usage, in-network ML update delays, and discusses their impact on latency-critical applications.

1) SRAM UTILIZATION

The SRAM consumption of our *Per-Flow Register Sketch* is analyzed by measuring the memory footprint under varying numbers of concurrent flows, as illustrated in Fig. 12. The baseline switch memory usage (with no active flows) is recorded at 18.69 MB. As flows are introduced, SRAM usage increased to 38.43 MB for 1K flows, 51.99 MB for 10K flows, and reached up to 188 MB for 100K flows. This linear scaling trend reflects the fixed per-flow register allocation inherent in our approach, wherein each flow is mapped to a set of registers storing key features such as maximum, minimum, and mean packet sizes. The steady increase in memory usage confirms that our method maintains a predictable and controllable overhead, rendering it suitable for mid-scale network deployments.

However, at 100K concurrent flows, the SRAM footprint reaches 188.81 MB, which could exceed the available memory in some commercial programmable switches [43], indicating a potential scalability limitation. This limitation becomes more critical in attack scenarios such as spoofing, where a massive number of fake flows could overwhelm the system. To address this issue, future research could explore adaptive memory management mechanisms to temporarily accommodate the increasing volume of flows and apply entropy-based anomaly detection to identify and mitigate spoofed traffic before it exhausts system resources.

2) IMPACT OF DECISION-TREE RULES ON MEMORY USAGE

The integration of decision tree based classifiers into the *MATs* of the PDP can present scalability challenges, particularly in terms of rule proliferation and memory utilization, if not appropriately optimized. For instance, the selection

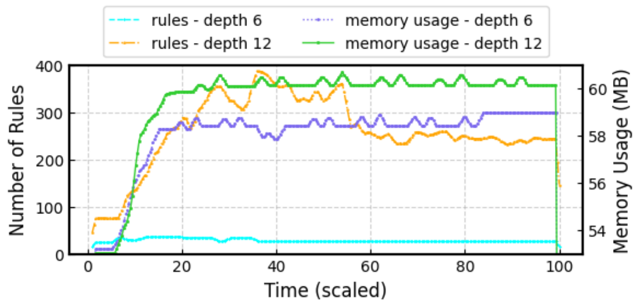

FIGURE 13. Impact of decision tree rules on memory usage.

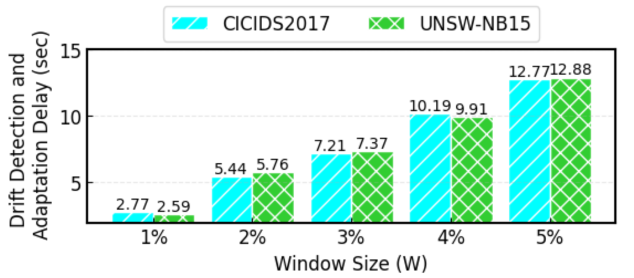

FIGURE 14. Drift detection and adaptation delay.

of an appropriate tree depth directly influences the switch's memory consumption. To assess this effect, we quantified the number of rules generated and the corresponding SRAM usage for decision trees with depths of 6 and 12. Given that BMv2 lacks TCAM memory, MATs reside in SRAM, rendering it the primary resource affected. Figure 13 illustrates these measurements, where the *x-axis* represents scaled time, capturing the progression of rule updates and memory usage over continuous retraining cycles. The left *y-axis* indicates the number of rules, while the right y-axis depicts SRAM usage (MB).

The results indicate that deeper trees generate significantly more rules, leading to increased memory consumption. The decision tree with depth 6 (cyan dashed line) maintains a relatively low rule count, stabilizing at approximately 50–100 rules, with a corresponding SRAM usage (purple dotted line) that remains below 56 MB. In contrast, depth 12 (orange dashed line) results in a substantially higher rule count, fluctuating between 250 and 350 rules, which causes a notable increase in memory consumption (solid green line), reaching nearly 60 MB. This observation confirms that deeper trees lead to an increased rule set, directly impacting the memory footprint in the PDP.

While this approach remains efficient for moderate-scale deployments, its scalability becomes a concern in large-scale scenarios where an extensive rule set must be maintained in SRAM. The exponential growth in rule entries with increasing tree depth can lead to memory exhaustion, limiting the feasibility of deeper decision trees in resource-constrained programmable switches. Additionally, in dynamic environments where rules must be updated frequently, handling a large *MAT* can introduce performance bottlenecks, affecting real-time packet processing.

To mitigate these challenges, lightweight ML alternatives such as binary tree classifiers or knowledge distillation techniques could be explored. A binary tree classifier provides a more memory-efficient representation of decision boundaries, reducing the number of rules stored in the data plane. Meanwhile, knowledge distillation enables a complex model to transfer knowledge to a simpler in-network model, balancing accuracy with resource constraints.

3) DRIFT ADAPTATION DELAY

Drift detection and adaptation delay is directly influenced by the *window size (W)*, as shown in Fig. 14. The results indicate that for both CICIDS2017 and UNSW-NB15, the delay increases as the window size grows. This is expected since larger windows require more samples before triggering adaptation, leading to longer waiting times. While the obtained delays are in the range of seconds, this is a direct result of the simulation setup, where the sender transmits batches of flows every second. Consequently, the system must wait for the buffer to fill before adaptation can occur.

However, in a real-time deployment, where flows arrive and are processed at microsecond or nanosecond levels, the adaptation mechanism would operate at significantly lower delays i.e., at micro-second level. This means that while the measured delay appears long under simulation constraints, the proposed approach is inherently adaptable to high-speed, real-time traffic processing without compromising responsiveness.

D. DISCUSSION ON REAL-WORLD APPLICATIONS

The proposed framework reflects real-world network security applications by addressing key challenges faced in modern programmable networks. Traditional IDS and intrusion prevention systems (IPS) rely on static models that struggle to adapt to evolving attack patterns, requiring frequent manual updates and introducing operational inefficiencies. In contrast, our method automates the detection and adaptation process, making it well-suited for deployment in enterprise networks, cloud data centers, and edge computing environments. By leveraging in-network ML within programmable switches, the framework enables real-time threat detection with minimal disruption to normal traffic, a critical requirement for service providers and enterprise IT infrastructures.

One of the most significant real-world applications of this method is in Industrial IoT (IIoT) environments and 5G network slicing security, where network conditions change dynamically due to varying traffic demands and emerging cyber threats. The framework's ability to monitor feature relevance and adjust classification rules ensures that security policies remain effective even as network behaviors evolve. In cloud-native security architectures, where virtualized workloads frequently migrate across different nodes, the ability to update attack detection models without reconfiguring the entire system is particularly advantageous. This allows cloud service providers to integrate adaptive security mechanisms into their Network Function Virtualization (NFV) and Software-Defined Networking (SDN) infrastructures, ensuring robust protection against volumetric attacks without significant manual intervention.

Nevertheless, the practical implementation of systems in real hardware environments presents challenges that are not present in simulation platforms like BMv2. Notably, although hardware switches function at line rate, the logging mechanisms and decision rule updates managed through the control plane are not instantaneous. These operations can introduce latency, which is often unacceptable in latency-sensitive applications, including industrial automation or real-time media streaming. Moreover, rule updates in hardware must be executed with precision to avoid inconsistencies or downtime. Unlike software-based switches, hardware switches typically necessitate hitless update strategies to prevent service disruption or packet loss during table modifications, conditions that are intolerable in mission-critical network environments. These deployment considerations underscore the necessity for efficient, low-latency coordination between the control and data planes to ensure that adaptive security mechanisms remain viable in production-grade, high-performance networks.

E. LIMITATIONS AND FUTURE DIRECTIONS

Although this study demonstrates adaptability to address traffic fluctuations, certain limitations and areas for further investigation persist. This subsection discusses key limitations and potential avenues for enhancing generalization, robustness, and efficacy of the system against more sophisticated attacks.

1) ENHANCING GENERALIZATION AND ADVERSARIAL RESILIENCE

The present study examines volumetric attacks, characterized by high-traffic anomalies that deviate markedly from typical network behavior. To assess the adaptability of the proposed framework, two widely used datasets, CICIDS2017 and UNSW-NB15, are selected, each exhibiting distinct attack characteristics. CICIDS2017 is characterized by high-rate periodic traffic bursts that simulate DoS conditions, whereas UNSW-NB15 encompasses more heterogeneous and stealthy attack patterns, such as those resulting from exploitation and fuzzing, which are dispersed and more challenging to detect. This contrast allows for the evaluation of the responsiveness of the system to various forms of drift, reflecting both abrupt and gradual shifts in malicious behavior. The performance observed across these datasets corroborates the adaptability of the framework to various traffic dynamics and validates its efficacy within this context.

However, these datasets primarily capture volumetric behaviors at the flow level and do not fully represent the broader spectrum of contemporary network threats. Specifically, multi-vector attacks, encrypted communication, and long-term stealthy intrusions - common in advanced persistent threats (APTs) - pose unique detection challenges. To enhance generalizability, future extensions of the framework could incorporate temporal and behavioral feature extraction techniques that model persistent anomalies over time, as well as methods suitable for encrypted traffic analysis.

In addition to broadening the coverage of advanced threat types, ensuring robustness against adversarial manipulation is essential for the resilience of adaptive systems. As drift-aware mechanisms become more prevalent, attackers may exploit them by injecting carefully crafted traffic patterns designed to trigger incorrect model updates or obscure malicious flows. One of the key challenges is balancing adaptation responsiveness with robustness: faster adaptation enables timely responses to legitimate shifts in traffic but risks overfitting to adversarial manipulations, such as mimicry attacks. Conversely, slower adaptation enhances resilience but may delay the detection of legitimate changes. Future research could explore techniques such as adversarial training, ensemble detection strategies, and selective or confidence-based adaptation to mitigate these risks. Furthermore, meta-learning approaches could be investigated to dynamically adjust adaptation speed and strategy based on the nature and confidence of observed drift, enabling the system to distinguish between benign and adversarial traffic evolution. Systematic evaluation under controlled adversarial conditions would be essential to validate the robustness of the framework and maintain high detection performance in evolving threat landscapes.

2) POTENTIAL MEMORY OPTIMIZATION STRATEGIES

The proposed framework utilizes a flow-register-based sketch with SRAM registers to maintain flow-level statistics within the data plane. Each incoming packet is hashed to a flow index based on its five-tuple, which corresponds to per-flow registers that store feature statistics such as maximum, minimum, and mean packet sizes. To optimize memory usage while maintaining detection accuracy, a summarized sketch such as the Count-Min Sketch (CMS) [44] can be employed. CMS is a compact, probabilistic data structure that facilitates approximate frequency estimation through hash-based counter arrays, providing substantial space savings at the expense of controlled estimation error. Building on this concept, Count-Less [45] introduces a layered sketch architecture with split counters and a minimum update strategy, dynamically assigning narrower counters to short-lived flows and reducing write contention by updating only the smallest among them. This design enhances memory efficiency under Zipfian traffic distributions. Conversely, SPArch [46] employs a pseudo-associative, multi-bucket layout with variable width counters, segregating flows by size class to minimize hash collisions and memory fragmentation. It also supports non-blocking updates and partial eviction, rendering it highly suitable for deployment in hardware-constrained, high-speed environments. These advanced sketch strategies offer promising avenues to improve the scalability and memory efficiency of our current register-based sketch design for real-world programmable switch and SmartNIC deployments.

3) SCALABILITY OVERHEAD IN HIGH-TRAFFIC ENVIRONMENTS

While the framework effectively adapts to traffic changes, frequent model updates introduce computational and network overhead, particularly in high-traffic conditions. Future research could investigate more efficient adaptation strategies, such as incremental learning or hierarchical drift handling, to identify false positives and further minimize unnecessary retraining cycles. Additionally, optimizing memory management and processing latency is essential for real-time deployment in high-speed networks. A comprehensive evaluation of the update frequency, resource consumption, and impact on classification performance will further enhance the scalability of the framework and its feasibility deployment in large-scale network environments.

4) RELIABILITY OF UNSUPERVISED LEARNING-BASED LABELING

The experimental findings suggest that unsupervised learning models demonstrate dynamic adaptation to novel traffic patterns; however, their efficacy remains significantly dependent on the characteristics of the dataset. The iForest algorithm is effective in detecting rare anomalies within heterogeneous environments (e.g., UNSW-NB15), but its accuracy diminishes when applied to structured periodic attack patterns (e.g., CICIDS2017). In contrast, k-Means clustering exhibits more stable performance across retraining cycles, making it more suitable for incremental adaptation. However, both methodologies are susceptible to labeling errors, particularly in high-traffic scenarios where attack spikes may closely resemble benign traffic patterns. These errors can propagate through retraining iterations, thus affecting the reliability of the model.

To mitigate such risks, future research could investigate validation mechanisms to assess and refine the quality of unsupervised labeling prior to model updates. For example, the Silhouette Score can quantify cluster cohesion and separation, providing a heuristic for label confidence. Furthermore, implementing confidence thresholding, where only data points distant from the cluster boundaries are selected for retraining, can help reduce the influence of ambiguous instances. Hybrid approaches, such as combining clustering with anomaly detection feedback loops or integrating weak supervision signals, could further enhance the robustness of the label. Ensuring adaptive tuning and incorporating these validation techniques will be essential for maintaining the integrity of pseudo-labels in large-scale, evolving network environments.

VI. CONCLUSION

This paper introduces an adaptive in-network defense method designed to protect networks from evolving volumetric attacks. We employ a continuous learning approach that leverages a data-driven method to establish baseline performance thresholds from historical data, using these thresholds as benchmarks for detecting anomalies in unseen data. By continuously updating these thresholds to reflect changes in data distribution and in-network inference results, our method efficaciously adapts to evolving attack patterns.

We validate the presence of drift in intrusion detection datasets and demonstrate its effect on model performance. Our evaluation shows that the adaptive nature of the ML model maintains classification accuracy despite evolving attacks. We also compare static and dynamic monitoring window sizes, finding that dynamic windows achieve better accuracy while reducing disruptions to normal network traffic. Our findings reveal a trade-off between drift detection accuracy and the effects of model adaptation. The adaptive adjustment of the monitoring window size helps manage this trade-off, providing a more responsive solution for detecting and adapting to new attack patterns.

REFERENCES

[1] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.

[2] R. Doshi, N. Apthorpe, and N. Feamster, "Machine learning DDoS detection for consumer Internet of Things devices," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2018, pp. 29–35.

[3] X. Chen et al., "Empowering DDoS attack mitigation with programmable switches," *IEEE Netw.*, vol. 37, no. 3, pp. 112–117, May 2023.

[4] P. Bosshart et al., "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, Jul. 2014.

[5] X. Chen et al., "Empowering network security with programmable switches: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1653–1704, 3rd Quart., 2023.

[6] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, Anaheim, CA, USA, 2023, pp. 6203–6220.

[7] B. M. Xavier, R. S. Guimarães, G. Comarela, and M. Martinello, "MAP4: A pragmatic framework for in-network machine learning traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4176–4188, Dec. 2022.

[8] K. Tgavalekos, J. M. Namayanja, and R. Alhassan, "Characterization of network behavior to detect changes: A cybersecurity perspective," in *Proc. Workshop Program 19th Int. Conf. Distrib. Comput. Netw.*, Mar. 2018, pp. 1–6.

[9] M. Amin, F. Al-Obeidat, A. Tubaishat, B. Shah, S. Anwar, and T. A. Tanveer, "Cyber security and beyond: Detecting malware and concept drift in AI-based sensor data streams using statistical techniques," *Comput. Electr. Eng.*, vol. 108, May 2023, Art. no. 108702.

[10] C. H. Tan, V. C. Lee, and M. Salehi, "MIR_MAD: An efficient and online approach for anomaly detection in dynamic data stream," in *Proc. Int. Conf. Data Mining Workshops (ICDMW)*, Nov. 2020, pp. 424–431.

[11] F. Gu, "Concept drift detection for machine learning with stream data," Ph.D. dissertation, Fac. Eng. Inf. Technol., Univ. Technol. Sydney, 2019.

[12] A. Kuppa and N.-A. Le-Khac, "Learn to adapt: Robust drift detection in security domain," *Comput. Electr. Eng.*, vol. 102, Sep. 2022, Art. no. 108239.

[13] G. Xie et al., "Empowering in-network classification in programmable switches by binary decision tree and knowledge distillation," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 382–395, Feb. 2024.

[14] B. M. Xavier, R. S. Guimar aes, G. Comarela, and M. Martinello, "Programmable switches for in-networking classification," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.

[15] X. Zhang, L. Cui, F. P. Tso, and W. Jia, "PHeavy: Predicting heavy flows in the programmable data plane," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4353–4364, Dec. 2021.

[16] F. Musumeci, A. C. Fidanci, F. Paolucci, F. Cugini, and M. Tornatore, "Machine-learning-enabled DDoS attacks detection in P4 programmable networks," *J. Netw. Syst. Manage.*, vol. 30, no. 1, pp. 1–27, Jan. 2022.

[17] B. Coelho and A. Schaeffer-Filho, "BACKORDERS: Using random forests to detect DDoS attacks in programmable data planes," in *Proc. 5th Int. Workshop P4 Eur.*, Dec. 2022, pp. 1–7.

[18] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "In-network machine learning using programmable network devices: A survey," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1171–1200, 2nd Quart., 2024.

[19] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.

[20] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Jun. 2015, pp. 1–6.

[21] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. V. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.

[22] O. Hireche, C. Benzaïd, and T. Taleb, "Deep data plane programming and AI for zero-trust self-driven networking in beyond 5G," *Comput. Netw.*, vol. 203, Feb. 2022, Art. no. 108668.

[23] M. Zhang et al., "Poseidon: Mitigating volumetric DDoS attacks with programmable switches," in *Proc. 27th Netw. Distrib. Syst. Secur. Symp. (NDSS)*, Apr. 2020.

[24] Z. Liu et al., "Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, May 2021, pp. 3829–3846.

[25] D. Ding, M. Savi, F. Pederzolli, M. Campanella, and D. Siracusa, "In-network volumetric DDoS victim identification using programmable commodity switches," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1191–1202, Jun. 2021.

[26] K. Friday, E. Bou-Harb, and J. Crichigno, "A learning methodology for line-rate ransomware mitigation with P4 switches," in *Proc. Int. Conf. Netw. Syst. Secur.* Cham, Switzerland: Springer, 2022, pp. 120–139.

[27] K. Friday, E. Kfoury, E. Bou-Harb, and J. Crichigno, "INC: In-network classification of botnet propagation at line rate," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2022, pp. 551–569.

[28] C. Zheng et al., "IIsy: Hybrid in-network classification using programmable switches," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2555–2570, Jun. 2024.

[29] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, and K. Ghédira, "Discussion and review on evolving data streams and concept drift adapting," *Evolving Syst.*, vol. 9, no. 1, pp. 1–23, Mar. 2018.

[30] L. Wang and R. Jones, "Big data analytics in cyber security: Network traffic and attacks," *J. Comput. Inf. Syst.*, vol. 61, no. 5, pp. 410–417, Sep. 2021.

[31] P. Bosshart et al., "Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 99–110, 2013.

[32] (Jan. 2024). *P4RuntimeAPI*. [Online]. Available: https://p4.org/p4-spec/p4runtime/main/P4Runtime-Spec.html

[33] N. K. Sharma et al., "Evaluating the power of flexible packet processing for network resource allocation," in *Proc. 14th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, May 2017, pp. 67–82.

[34] J. Gama, P. Medas, G. Castillo, and P. P. Rodrigues, "Learning with drift detection," in *Proc. 17th Brazilian Symp. Artif. Intell. Adv. Artif. Intell. (SBIA)*, Sep. 2004, pp. 286–295.

[35] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," *IEEE Access*, vol. 9, pp. 87094–87155, 2021.

[36] M. Sundararajan and A. Najmi, "The many Shapley values for model explanation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2020, pp. 9269–9278.

[37] Z. Xiong and N. Zilberman, "Do switches dream of machine learning? Toward in-network classification," in *Proc. 18th ACM Workshop Hot Topics Netw.*, Nov. 2019, pp. 25–33.

[38] M. Saqib, Z. A. Hmitti, H. Elbiaze, and R. H. Glitho, "An accurate & efficient approach for traffic classification inside programmable data plane," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2022, pp. 6331–6336.

[39] K. P. Sinaga and M.-S. Yang, "Unsupervised K-means clustering algorithm," *IEEE Access*, vol. 8, pp. 80716–80727, 2020.

[40] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, Jun. 2008, pp. 413–422.

[41] (Jan. 2024). *Performance of BMV2*. [Online]. Available: https://github.com/p4lang/behavioral-model/blob/main/docs/performance.md#what-impacts-performance

[42] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[43] Barefoot Networks. (2024). *Tofino Programmable Switch*. Accessed: Feb. 2025. [Online]. Available: https://www.barefootnetworks.com/

[44] G. Cormode, "Count-min sketch," in *Encyclopedia of Database Systems*. Boston, MA, USA: Springer, 2009, pp. 511–516.

[45] S. Kim, C. Jung, R. Jang, D. Mohaisen, and D. Nyang, "Count-less: A counting sketch for the data plane of high speed switches," 2021, *arXiv:2111.02759*.

[46] A. Sateesan, J. Vliegen, S. Scherrer, H.-C. Hsiao, A. Perrig, and N. Mentens, "SPArch: A hardware-oriented sketch-based architecture for high-speed network flow measurements," *ACM Trans. Privacy Secur.*, vol. 27, no. 4, pp. 1–34, Nov. 2024.

MUHAMMAD SAQIB (Graduate Student Member, IEEE) received the master's degree in computer science from the University of Engineering and Technology, Taxila, Pakistan, in 2019. He is currently pursuing the Ph.D. degree in computer science with the Université du Québec à Montréal (UQAM), Montreal, Canada. His research interests include traffic classification and quality of service management in next generation networks.

HALIMA ELBIAZE (Senior Member, IEEE) received the Ph.D. degree in computer science from Télécom SudParis, France, in 2002. Since 2003, she has been with the Department of Computer Science, Université du Québec à Montréal, Montreal, QC, Canada, where she is currently a Full Professor. Her current research interests include network performance evaluation, traffic engineering, and quality of service management in next generation networks.

ROCH H. GLITHO (Life Senior Member, IEEE) has held a Canada Research Chair from 2010 to 2020. Prior to joining academia in 2010, he has held several senior technical positions at Ericsson. He is currently a Full Professor with Concordia University, where he holds the Ericsson/ENCQOR Industrial Research Chair in Cloud/Edge for 5G and Beyond. He is also a Professor Extraordinaire with the University of Western Cape, South Africa.